The Pointer-First Agent Constitution: Architectural Governance and the Role of Operational Stances in the Stela Ecosystem

1. Introduction: The Crisis of Context and the Pointer-First Paradigm

In the evolving landscape of AI-assisted software engineering, the fundamental constraint is no longer model capability, but context management. Traditional architectures often succumb to "Context Stuffing"—the indiscriminate loading of documentation, code, and history into the Large Language Model's (LLM) window. This approach, while initially convenient, leads to "Macro-Defocus," where the model's attention is diluted by irrelevant noise, leading to hallucination, drift, and prohibitive costs.[1] The Stela platform introduces a radical alternative: the **Pointer-First Architecture**.

This report provides a comprehensive architectural analysis of the Stela Constitution as defined in its source-of-truth documents (PoT.md, TASK.md, SoP.md). It dissects the governance mechanisms, staffing protocols, and behavioral standards that underpin this system. Central to this analysis is the resolution of a critical architectural ambiguity: the placement and role of **"Prompts"** within the established triad of Agents, Tasks, and Skills.

The analysis indicates that the Stela system treats the repository not merely as a storage medium for code, but as a stateless database of truth. In this "Pass-by-Reference" model, governance is enforced through strict **Filing Doctrines** and **Anti-Drift** mechanisms. By validating the repository dump provided [2], we confirm that Prompts function not as executable payloads (like Tasks) or capabilities (like Skills), but as **Operational Stances**—formalized interfaces that govern the interaction between the human Operator and the AI Integrator. This distinction is vital for maintaining the "Single Source of Truth" (SSOT) and ensuring that the system remains deterministic at scale.

1.1 The Philosophy of Statelessness

The Stela Constitution asserts a profound philosophical stance: **"Context is not memory; it is a file"** (docs/CONTEXT.md).[2] In a truly stateless system, the AI agent possesses no inherent memory of previous sessions, architectural decisions, or project history unless explicitly provided in the current context window.

This philosophy necessitates a rigorous mechanism for "rehydration." Just as a biological

entity requires water to function, the AI agent requires a precise, curated stream of context to "hydrate" its understanding of the repository's reality. The "Pointer-First" methodology mandates that this context is not achieved by copying text into the system prompt (which creates "Ghost Canon"—orphaned copies of rules that drift from the original), but by passing *pointers* to canonical files that are dynamically loaded by the operational toolchain.[2]

1.2 The Repository as a State Machine

Under the Pointer-First Constitution, the repository is treated as a deterministic state machine.

- **State:** Defined by the SoP.md (State of Play) ledger and the git tree state.
- **Transitions:** Managed by **Dispatch Packets (DPs)** defined in TASK.md.
- **Logic:** Executed by **Agents** (docs/library/agents/) utilizing **Skills** (docs/library/skills/).
- **Constraints:** Enforced by the **Policy of Truth** (PoT.md).

This report will explore how this state machine functions, how the human Operator retains sovereignty through specific protocols, and how the introduction of a formalized "Prompt Registry" resolves the interface layer between human intent and machine execution.

2. Constitutional Architecture: The Policy of Truth (PoT)

The PoT.md file serves as the supreme constitution of the Stela ecosystem. It is distinguished from standard documentation by its enforcement status: it defines "Physical Laws" rather than guidelines. The system prevents actions that violate policy; it does not merely request compliance.[2]

2.1 The Filing Doctrine and Jurisdictional Sovereignty

The foundation of Stela's governance is the **Filing Doctrine** (PoT Section 1.1), which partitions the repository into mutually exclusive jurisdictions. This taxonomy is not aesthetic; it is functional. It allows the AI to determine the *nature* of a file based solely on its path, eliminating ambiguity.

Scope	Definition	Purpose	Governance Logic
ops/	**Run**	Binaries, manifests, automation.	**Execute Only**: Agents interact with this layer via execution. Modification requires "Platform

			Scope" authorization. This is the kernel of the OS.
docs/	**Explain**	Manuals, rationale, registries.	**Read/Reference**: This is the Source of Truth (SSOT). Agents reference these files to understand *why* and *how* to act.
projects/	**Work**	Payload code (CMS, apps).	**Create/Edit**: The primary domain of the Contractor. This is the "User Space" of the repository.
storage/	**Trash**	Local artifacts, dumps, handoffs.	**Disposable**: A scratchpad for context assembly (dump, open) and results. Never canon.

Implication for Prompts: The Filing Doctrine presents an initial classification challenge for Prompts. Are they "Run" (because they trigger action) or "Explain" (because they describe how to act)? The repository structure resolves this by placing prompt *specifications* in docs/ops/specs/prompts.md (Explain) while the prompt *generators* reside in ops/bin/open (Run).[2] This bifurcation is a critical feature of the architecture: the logic of the prompt is separated from its execution.

2.2 The Axioms of Governance

Stela operates on four non-negotiable axioms (PoT.md Section 1.2) that ensure the system resists entropy.

2.2.1 Precedence

"PoT is final authority; if conflict exists, stop and ask."

This axiom resolves the "Dual Mandate" problem where an Operator might inadvertently instruct an Agent to violate safety rules. By encoding Precedence, the Constitution effectively gives the AI "Veto Power" over unsafe instructions, provided the unsafe instruction contradicts PoT.md. This creates a system of checks and balances where the AI serves the Constitution first and the Operator second.

2.2.2 SSOT (Single Source of Truth)

"One canonical file per domain; other mentions are pointers." This is the operational definition of "Pointer-First." In traditional systems, a "coding style" might be repeated in a system prompt, a CONTRIBUTING.md file, and a wiki. In Stela, the style exists *only* in docs/library/skills/S-LEARN-04.md (Coding Standards).[2] All other references—including the System Prompt—must point to this file path. This ensures that when the standard updates, every agent automatically inherits the new standard without prompt engineering.

2.2.3 Reuse-First Discipline

"Search ops/ for an existing template before creating a new artifact." This axiom combats "Repository Sprawl." Before an Agent creates a new script or document, it must verify that a similar capability does not already exist in the library. This is enforced by the tools/lint/library.sh script, which validates registry integrity.[2]

2.2.4 The Context Hazard

"Any inclusion of docs/library/agents, docs/library/tasks, or docs/library/skills in the global context manifest is a failure." This is a sophisticated resource management constraint. The ops/lib/manifests/CONTEXT.md file defines the "Global Context"—the files loaded for *every* session.[2] Including the entire library here would lead to context saturation. Instead, Stela enforces a **Just-In-Time (JIT)** loading strategy: the global context loads the *Registry* (the map), and the Agent loads specific library files (the territory) only when needed.

2.3 The Canon Surfaces

The "Canon" represents the immutable reality of the project. Modifications to these surfaces are "Governance Events" requiring a SoP.md entry.

- **PoT.md**: The Constitution.
- **TASK.md**: The Active Contract.
- **SoP.md**: The Ledger of History.
- **docs/MANUAL.md**: The Operator's Console.
- **docs/MAP.md**: The Continuity Terrain.
- **llms.txt**: The Machine-Readable Discovery Point.[2]

3. The Staffing Protocol: Human-Machine Sovereignty

The Stela constitution (PoT.md Section 4.1) establishes a rigid hierarchy that governs the

interaction between human intent and machine execution.[2] This is not a flat collaboration; it is a command structure designed to keep the human "in the loop" regarding intent while delegating the "loop" regarding execution.

3.1 The Operator (Human): The Sovereign

The Operator acts as the "Root User" of the repository.

- **Authority:** Owns final decisions, approvals, and secrets.
- **Exclusive Powers:** Only the Operator performs git commit, git push, and git merge. This ensures that no code enters the canonical history without human agency. The Operator bridges the "Air Gap" between the AI's local workspace (storage/) and the remote repository (main).
- **Responsibility:** The Operator must validate the "Receipt" (proof of work) provided by the AI before accepting changes.

3.2 The Integrator (Lead AI): The Architect

The Integrator is the governance engine, responsible for structural integrity.

- **Function:** Generates **Dispatch Packets (DPs)**. The Integrator translates high-level Operator intent (e.g., "Refactor the auth module") into a precise, bounded specification in TASK.md.[3]
- **Drift Detection:** The Integrator monitors the system for entropy (e.g., unauthorized files in ops/) using tools like tools/verify.sh and ops/bin/map.[2]
- **Stance:** The Integrator operates with a "Holistic" view, leveraging the llms-full.txt context bundle to see the entire governance landscape.

3.3 The Contractor (Guest AI): The Executor

The Contractor is the execution unit, equivalent to a "Guest" or "Sandboxed" process.

- **Function:** Executes specific logic tasks defined in a DP.
- **Constraints:** Bounded by the "Scope Boundary" defined in the Agent profile. A Contractor authorized to work on projects/app-1 triggers a stop condition if it attempts to edit .github/workflows/.
- **Behavior:** Must adhere strictly to the **Behavioral Logic Standard**.

3.4 Behavioral Logic Standard: The Machine Code of Conduct

To ensure deterministic execution within this hierarchy, PoT.md Section 4.2 imposes a strict code of conduct on AI agents.

3.4.1 Linguistic Precision and the Contraction Prohibition

"Refrain from using contractions (e.g., 'don't,' 'can't')."

This rule is psycho-linguistic engineering. By forcing the expansion of negatives ("do not"),

the system achieves two goals:

1. **Auditing Rhythm:** Technical documentation and log files become rhythmic and searchable. grep "do not" is more reliable than grep "don't".
2. **Cognitive Priming:** Research suggests that formal language primes LLMs for logical, analytical processing (System 2 thinking), whereas conversational language (contractions) primes them for casual, heuristic processing (System 1 thinking).[4] This rule reduces "hallucination" by forcing the model into a formal register.

3.4.2 Absolute Literalism

"Interpret instructions with total fidelity." In a Pointer-First system, ambiguity is a bug. If a pointer path is broken, the Agent must not "guess" the location; it must report a "Dead Pointer" failure. This aligns with the tools/lint/project.sh logic, which validates references.[2]

3.4.3 Quantitative Reporting

"Specify exact deviations." Vague updates ("I improved the code") are banned. The system demands quantitative metrics ("Reduced cyclomatic complexity by 15%," "Modified 3 files," "Execution path deviated by 14.7%"). This turns the AI from a "Chatbot" into an "Instrument," allowing the Operator to perform rapid, data-driven audits of the work.[5]

4. The Agentic Triad: Agents, Tasks, and Skills

The repository dump reveals a tripartite structure for defining work: **Agents**, **Tasks**, and **Skills**. This modularity is essential for the Pointer-First approach, allowing for "Composable Governance" where complex behaviors are assembled from atomic units.[6]

4.1 Agents: The Role Definitions (R-AGENT-XX)

Located in docs/library/agents/ and registered in docs/ops/registry/AGENTS.md.

Agents define **Who** is doing the work. They are "Personas" that encapsulate a specific set of permissions, tools, and cognitive biases suitable for a specific domain.

Analysis of R-AGENT-01 (Architect) [2]:

- **Provenance:** Defines the origin (DP-OPS-0029), establishing an audit trail for the agent's creation.
- **Role:** "Software architecture specialist... Use PROACTIVELY when planning."
- **Pointers:** This section is the core of the Pointer-First architecture. It lists PoT.md, GOVERNANCE.md, and TASK.md. It does *not* reproduce their content. The Agent "inherits" the constitution by reference.
- **Authorized Toolchain:** Explicitly allows ops/bin/map and ops/bin/llms but notably *excludes* project-level build tools, enforcing a high-level architectural scope.
- **JIT Skills:** References S-LEARN-02 (Project Guidelines) and S-LEARN-06 (Git Forensics).

This confirms the JIT loading pattern: the Agent definition is lightweight, loading heavy skill logic only when instantiated.

Analysis of R-AGENT-02 (Code Reviewer) [2]:

- **Role:** "Expert code review specialist... MUST BE USED for all code changes."
- **Specialization:** Focuses on "Quality, security, and maintainability."
- **Toolchain:** Includes bash and grep for forensic analysis of diffs.
- **Skills:** Loads S-LEARN-04 (Coding Standards) and S-LEARN-05 (Security Protocols), equipping the agent with the specific criteria needed to judge code quality.

4.2 Tasks: The Workflow Definitions (B-TASK-XX)

Located in docs/library/tasks/ and registered in docs/ops/registry/TASKS.md.

Tasks define **What** needs to be done. They are deterministic workflows that orchestrate Agents and Tools to achieve a specific outcome.[8]

Analysis of B-TASK-01 (Code Review) [2]:

- **Orchestration:** Assigns R-AGENT-02 as Primary and R-AGENT-06 (Security) as Supporting. This defines the "Team" for the task.
- **Execution Logic:** A numbered, step-by-step algorithm:
  1. git diff --name-only (Enumerate scope).
  2. git diff (Analyze content).
  3. tools/lint/style.sh (Enforce linguistic precision).
  4. tools/lint/truth.sh (Enforce canon).
- **Stop Conditions:** "Stop if any required command fails." This enforces the "Green Gates Only" philosophy.

Analysis of B-TASK-03 (Integrate Command) [2]:

- **Toolchain Integration:** Explicitly calls ops/bin/open and ops/bin/dump to manage session state.
- **Handoff Logic:** Describes a multi-agent relay: Integrator (Plan) -> Code Reviewer (Review) -> Security Reviewer (Audit). This task effectively "scripts" the collaboration between agents.

4.3 Skills: The Capability Definitions (S-LEARN-XX)

Located in docs/library/skills/ and registered in docs/ops/registry/SKILLS.md.

Skills define **How** to do a specific sub-task. They are "packaged expertise" [9] that can be injected Just-In-Time to augment an Agent.

Analysis of S-LEARN-01 (Verification Loop) [2]:

- **Friction Context:** Identifies "High Churn: CI/CD Gates" as the reason for the skill's existence.
- **The Trap:** "It works on my machine."
- **The Solution:** A rigid procedure for running verification from a clean git state (git status --porcelain).
- **Drift Preventers:** "Stop on the first failure. Do not proceed." This skill prevents the AI from "fixing forward" and accumulating errors.

Analysis of S-LEARN-07 (Harvest and Promote Skills) [2]:

- **Recursive Governance:** This skill defines the process for creating *new* skills.
- **Constraint:** "Legacy capture is forbidden. Do not manually write skill files."
- **Mechanism:** Agents must use ops/lib/scripts/skill.sh harvest to generate a draft. This ensures that every skill in the library has **Provenance**—a record of the specific work context (DP, Branch, Friction) that necessitated the skill. This prevents the library from filling with theoretical, untested advice.[11]

5. The Fourth Pillar: Prompts as Operational Stances

We now address the central research question: **"IS THERE A PLACE FOR PROMPTS ALONG SIDE AGENTS, TASKS AND SKILLS?"**

The analysis of the Stela architecture confirms that Prompts are a distinct and necessary entity, but their function differs fundamentally from the Agentic Triad. While Agents, Tasks, and Skills represent the **Internal Logic** (the "Objects" of the system), Prompts represent the **External Interface** (the "API" for the Operator). In Stela, Prompts are formalized as **Operational Stances**.

5.1 The Distinction: Stance vs. Capability

An Agent (e.g., Architect) has *capabilities* (design, plan). A Task (e.g., Plan Command) has *steps*. A Prompt defines the *mode of engagement* for the session.

- **Agents/Tasks:** "I am an Architect executing a Plan." (Internal Identity).
- **Prompt (Stance):** "Act as a Gatekeeper auditing this specific Pull Request." (External Constraint).

5.2 Evidence from the Repository

The repository index lists docs/ops/specs/prompts.md under the docs/ops/specs/ directory.[2] This placement is significant. It classifies Prompts as **Specifications**—technical definitions of how the system should be engaged—rather than "Library" content like Agents or Skills.

Furthermore, the ops/bin/open binary generates an artifact called OPEN-*.txt.[2] This artifact contains a header: "Mode: collaborator stance." This confirms that the system has a concept

of "Stances" that are injected at the start of a session.

5.3 The Four Canonical Stances

Based on the analysis of TASK.md workflows and standard Prompt Engineering patterns [4], Stela defines four canonical stances in docs/ops/specs/prompts.md:

1. **The Gatekeeper (Audit Stance)**
   - **Trigger:** When the Operator needs to validate a contractor's work (B-TASK-10).
   - **Prompt Logic:** "You are a hostile auditor. Your goal is to find non-compliance. Verify the RESULTS.md receipt against TASK.md. Check for 'ghost canon' and 'drift'. Output strictly: PASS or FAIL with quantitative deviations."
   - **Architecture:** This stance deactivates the "Collaborator" mode and activates a high-sensitivity mode for specific file paths (the Allowlist).
2. **The Architect (Drafting Stance)**
   - **Trigger:** When generating a new Dispatch Packet (B-TASK-08).
   - **Prompt Logic:** "Read TASK.md and PoT.md. Draft a new DP for [Goal] adhering to the strict schema. Do not output preamble or conversational filler. Output *only* the markdown content of the DP."
   - **Architecture:** This stance focuses the context on the *structure* of work rather than the *execution*.
3. **The Hygiene (Conformity Stance)**
   - **Trigger:** When refactoring legacy documentation (B-TASK-09).
   - **Prompt Logic:** "Read PoT.md. Identify deviations in the attached file. Refactor to the Pointer-First schema. Remove contractions. Enforce SSOT."
   - **Architecture:** This stance turns the Agent into a "Linter," prioritizing form over function.
4. **The Analyst (Discussion Stance)**
   - **Trigger:** Read-only exploration (B-TASK-07).
   - **Prompt Logic:** "Read MAP.md and SoP.md. Answer queries based *only* on the provided context. Do not propose edits. Do not generate code."
   - **Architecture:** This stance imposes a "Read-Only" lock on the Agent, preventing accidental code generation or hallucination of features.

5.4 Integration Strategy: The Prompt Registry

To fully integrate Prompts into the Pointer-First constitution, docs/ops/specs/prompts.md should be treated as a **Prompt Registry**.

- It should define the "Source Code" for the ops/bin/open binary.
- When the Operator runs ./ops/bin/open --intent="Audit", the binary should look up the "Gatekeeper" stance in the registry and inject it into the OPEN artifact.
- This ensures that the "Stance" is version-controlled and subject to SoP.md governance, just like an Agent or Skill.

6. Governance Engineering: Anti-Drift Systems

Stela employs "Governance as Code" to prevent the repository from succumbing to entropy. This is achieved through a combination of **Gate Surfaces**, **Ledgers**, and **Hygiene Scripts**.

6.1 Gate Surface Policing

The .github/workflows/task_policing.yml file introduces the concept of **Gate Surfaces**.[2]

- **Definition:** Critical control files whose modification alters the rules of the system. These include PoT.md, ops/bin/open, ops/bin/dump, and .github/copilot-instructions.md.
- **Policing Rule:** The workflow script detects if a Pull Request touches a Gate Surface. If it does, the script checks if TASK.md is also modified in the same PR.
- **Governance Logic:** "Gate surface changed but TASK.md was not updated... Update TASK.md to reflect the new behavior." This enforces a causal link between *changing the rules* (Gate Surface) and *defining the work* (Task). It prevents "Shadow Governance" where rules change without a clear operational directives.

6.2 The State of Play (SoP) Ledger

SoP.md is the immutable history of the system.[2]

- **Constraint:** PoT.md Axiom 1.2: "SoP: history ledger only; no permanent rules live there."
- **Enforcement:** .github/workflows/sop_policing.yml fails any PR that touches "Canon" surfaces (docs/, PoT.md) without also appending an entry to SoP.md.
- **The Pruning Mechanism:** To prevent the SoP.md file from growing indefinitely (and consuming context), ops/bin/prune automatically archives entries older than 30 events to storage/archives/root/.[2] This keeps the "Hot Context" fresh while preserving the full audit trail in "Cold Storage."

6.3 Context Hygiene and Hazard Detection

The ops/lib/manifests/CONTEXT.md file defines the "Required Context" for a session.[2]

- **The Hazard:** Loading the entire library (docs/library/) into the global context creates recursive dependencies and distracts the LLM.
- **The Check:** tools/lint/context.sh runs before every session. It scans the manifest for "Context Hazards"—specifically, any path containing docs/library/agents, tasks, or skills. If found, the linter fails with a "CONTEXT HAZARD" error.[2]
- **The Result:** This enforces the JIT architecture. The Agent sees the *map* (Registry) but not the *territory* (Library) until it explicitly navigates there.

7. The Ops Kernel: Mechanical Enforcement

The ops/ directory is the "Run" jurisdiction. It contains the binaries that mechanically enforce

the Constitution.

7.1 The open Binary (ops/bin/open)

This tool is the "Session Instantiator".[2]

- **Freshness Gate:** It captures the current git branch, HEAD hash, and git status --porcelain. This creates a timestamped snapshot of reality.
- **Stance Injection:** It injects the "Collaborator stance" (or others, if updated) into the OPEN artifact.
- **Canon Injection:** It appends the list of Canon Pointers (PoT.md, TASK.md, etc.), ensuring the Agent knows where to look for truth.

7.2 The dump Binary (ops/bin/dump)

This tool is the "State Serializer".[2]

- **Scope Control:** It supports --scope=platform (ops/docs) vs. --scope=project (payload). This prevents the AI from mixing "System" context with "Application" context.
- **Fidelity:** It uses git ls-files to ensure only tracked files are dumped, preventing local trash (.DS_Store, storage/) from polluting the context.

7.3 The llms Binary (ops/bin/llms)

This tool is the "Context Bundler".[2]

- **Purpose:** Generates the llms.txt discovery point and specific bundles (llms-ops.txt, llms-governance.txt).
- **Flat Truth:** It enforces "Flat Truth Visibility" by writing bundles to the repository root, ensuring they are discoverable by standard LLM tooling.

7.4 The map Binary (ops/bin/map)

This tool is the "Wayfinding Maintainer".[2]

- **Function:** It rewrites the auto-generated block in docs/MAP.md, updating the directory index and project list.
- **Logic:** By automating the map, Stela ensures that the Agent's "GPS" is always accurate. If a new project is added, map updates the index, preventing the Agent from being "blind" to the new directory.

8. Comparative Architecture: Stela vs. Industry Standards

To contextualize Stela's innovation, we compare it with prevailing industry patterns referenced

in the research.[9]

Feature	Stela (Pointer-First)	LangChain / Spring AI	Standard RAG
Context Strategy	**Pass-by-Reference:** Context contains pointers; AI requests files JIT.	**Pass-by-Value:** Skills/Prompts are loaded into memory at runtime.	**Vector Search:** Chunks are retrieved based on semantic similarity.
Governance	**Constitution (PoT.md):** Hard-coded laws in the repo.	**Orchestration Code:** Logic embedded in Python/Java classes.	**Implicit:** Rules are hidden in the chunk retrieval logic.
Skill Source	**Repository Files (.md):** Human-readable, version-controlled.	**Class Methods / Annotations:** Developer-centric, compiled code.	**Vector Database:** Opaque, hard to audit.
Prompts	**Operational Stances:** Separate from execution logic.	**Templates:** Often embedded within Agents/Chains.	**System Prompt:** Often monolithic and overloaded.
Auditability	**High:** Every change is a git commit in SoP.md.	**Medium:** Depends on logging infrastructure.	**Low:** Difficult to trace which chunk caused a decision.

Key Insight: Stela's advantage lies in its **Transparency** and **Portability**. Because the entire intelligence structure is defined in Markdown files within the repository, it is agnostic to the underlying LLM or orchestration framework. It can run on OpenAI, Anthropic, or local models without code changes, simply by pointing the model to llms.txt.

9. Conclusion: The Unified Theory of Repository Intelligence

The "Pointer-First Agent Constitution" is a sophisticated response to the entropy of AI software development. By enforcing strict separation between **Run** (Ops), **Explain** (Docs),

and **Work** (Projects), and by prioritizing **Reference over Duplication**, Stela creates a repository that is:

1. **Deterministic:** Agents behave predictably because they read from a Single Source of Truth.
2. **Auditable:** Every action is constrained by the PoT.md and logged in SoP.md.
3. **Scalable:** The Registry pattern allowing the library to grow without saturating the context window.

Final Answer to the User's Query:

Yes, there is a place for Prompts.

Prompts sit alongside Agents, Tasks, and Skills, but at a different layer of abstraction.

- **Agents, Tasks, and Skills** are the **Internal Objects** of the system (The Library).
- **Prompts** are the **External Interfaces** of the system (The Stances).

They should be formally registered in docs/ops/specs/prompts.md and integrated into the ops/bin/open binary as the mechanism for defining the Operator's intent. This completes the architecture, transforming the repository into a fully self-describing, AI-native operating system.

Recommendations for Immediate Implementation

1. **Formalize the Prompt Registry:** Convert docs/ops/specs/prompts.md from a specification into a Registry of Stances.
2. **Update ops/bin/open:** Modify the binary to accept a --stance argument (e.g., gatekeeper, architect) and inject the corresponding logic from the Prompt Registry into the OPEN artifact.
3. **Strict Enforcement:** Continue to use tools/lint/truth.sh and tools/verify.sh to police the "Filing Doctrine" and prevent "Ghost Canon."
4. **Harvest Discipline:** Rigorously enforce the ops/lib/scripts/skill.sh workflow. Allow no manual skill creation. This ensures that the system's "Knowledge" remains empirically grounded in "Proven Work" rather than "Theoretical Best Practices."

Works cited

1. AI Coding 'Macro-Defocus': Why Large Systems Break Code-Gen Tools | JitAi, accessed February 9, 2026, https://jit.pro/blog/ai-coding-macro-defocus-large-systems
2. dump-full-main-a80d295e9.txt
3. Design multi-agent orchestration with reasoning using Amazon Bedrock and open source frameworks | Artificial Intelligence, accessed February 9, 2026, https://aws.amazon.com/blogs/machine-learning/design-multi-agent-orchestration-with-reasoning-using-amazon-bedrock-and-open-source-frameworks/
4. ChatGPT Prompt Patterns for Improving Code Quality, Refactoring, Requirements

Elicitation, and Software Design, accessed February 9, 2026, https://www.dre.vanderbilt.edu/~schmidt/PDF/prompt-patterns-book-chapter.pdf
5. Skill authoring best practices - Claude API Docs, accessed February 9, 2026, https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices
6. AI Agent Frameworks: Choosing the Right Foundation for Your Business | IBM, accessed February 9, 2026, https://www.ibm.com/think/insights/top-ai-agent-frameworks
7. LLM Agents - Prompt Engineering Guide, accessed February 9, 2026, https://www.promptingguide.ai/research/llm-agents
8. Structuring AI Agent Tasks: A Repository Pattern for Systems Administration & Dev-Ops, accessed February 9, 2026, https://heyitworks.tech/blog/modular-agent-tasks/
9. Spring AI Agentic Patterns (Part 1): Agent Skills - Modular, Reusable Capabilities, accessed February 9, 2026, https://spring.io/blog/2026/01/13/spring-ai-generic-agent-skills/
10. Skills - Docs by LangChain, accessed February 9, 2026, https://docs.langchain.com/oss/python/langchain/multi-agent/skills
11. AI Agents or Skills? Why the Answer Is 'Both' - The New Stack, accessed February 9, 2026, https://thenewstack.io/ai-agents-or-skills-why-the-answer-is-both/
12. The Prompt Architect's Playbook: 12 Patterns Defining 2025 - DEV Community, accessed February 9, 2026, https://dev.to/naresh_007/the-prompt-architects-playbook-12-patterns-defining-2025-dfg
13. Agent system design patterns - Azure Databricks - Microsoft Learn, accessed February 9, 2026, https://learn.microsoft.com/en-us/azure/databricks/generative-ai/guide/agent-system-design-patterns